SIMATS ENGINEERING

ASSESSMENT TOOL 1

Constraint Based Problem Solving – Level 1

Course Name	R Programming
Course Code	ITA04
Course Outcome	Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively.
Student Name	Sameer Ahmed G
Register Number	192421154
Assessment Weightage	20%
Questions	5
Total Marks	100

Student Declaration

I, Sameer Ahmed G (Register No. 192421154), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sameer Ahmed G

Assessment Overview

This assessment demonstrates the application of fundamental R programming concepts through five constraint-based problems. The solutions are designed to satisfy the stated restrictions while demonstrating clear problem understanding, algorithmic reasoning, appropriate R constructs, pseudocode, complete implementation, and verification through outputs.

Approach Used to Meet the Exemplary Criteria

- Problem Understanding: Inputs, required outputs, constraints, and the relevant R data structure are explicitly identified for every question.
- Algorithm Design: Each problem contains a complete step-by-step algorithm that directly maps the requirements to R operations.
- Use of Control Structures: Solutions use appropriate base R operations, logical indexing, conditional assignment, vectorized operations, and apply() where specified, while avoiding prohibited loops.
- Pseudocode Structure: Each solution includes concise, ordered pseudocode before the R implementation.
- Completeness: Every sub-task (a)–(d) is addressed, expected results are stated, and the code follows the constraints without unnecessary constructs.

Question 1: Vectors – Weekly Sales Analysis

1.1 Problem Understanding

A shop records the number of units sold on each day of a week. The values are Monday = 12, Tuesday = 18, Wednesday = 9, Thursday = 25, Friday = 30, Saturday = 14, and Sunday = 22.

Input: Seven daily sales values with their corresponding day names.

Output: A named sales vector, total sales, weekly average rounded to two decimal places, days whose sales exceed the average, and the day with the highest sales.

Constraints: A single named vector must be created using c(); no for, while, or repeat loops may be used; logical indexing must be used for above-average days; the average must be rounded to two decimal places.

1.2 Algorithm

- Create a named vector called sales using c() with the seven daily sales values.
- Calculate total weekly sales using sum(sales).
- Calculate the weekly average using mean(sales) and round it to two decimal places.
- Use logical indexing sales > average_sales to select only values above the weekly average.
- Use which.max(sales) to identify the position of the maximum sales value and names() to obtain the corresponding day.
- Display all required results.

1.3 Pseudocode

```
START
Create named vector sales with seven daily values
total_sales ← sum of sales
average_sales ← round(mean of sales, 2)
above_average ← sales where sales > average_sales
highest_day ← name at position of maximum sales
Display sales, total, average, above_average, highest_day
STOP
```

1.4 R Program

```
# Question 1: Weekly Sales Analysis

sales <- c(
  Monday = 12,
  Tuesday = 18,
  Wednesday = 9,
  Thursday = 25,
  Friday = 30,
  Saturday = 14,
  Sunday = 22
)

print(sales)

total_sales <- sum(sales)
average_sales <- round(mean(sales), 2)

cat("Total sales:", total_sales, "\n")
cat("Average sales:", average_sales, "\n")

above_average <- sales[sales > average_sales]

cat("Days with sales above average:\n")
print(above_average)

highest_day <- names(sales)[which.max(sales)]

cat("Day with highest sales:", highest_day, "\n")
```

1.5 Expected Output

```
  Monday   Tuesday Wednesday  Thursday    Friday  Saturday    Sunday
      12        18         9        25        30        14        22

Total sales: 130
Average sales: 18.57

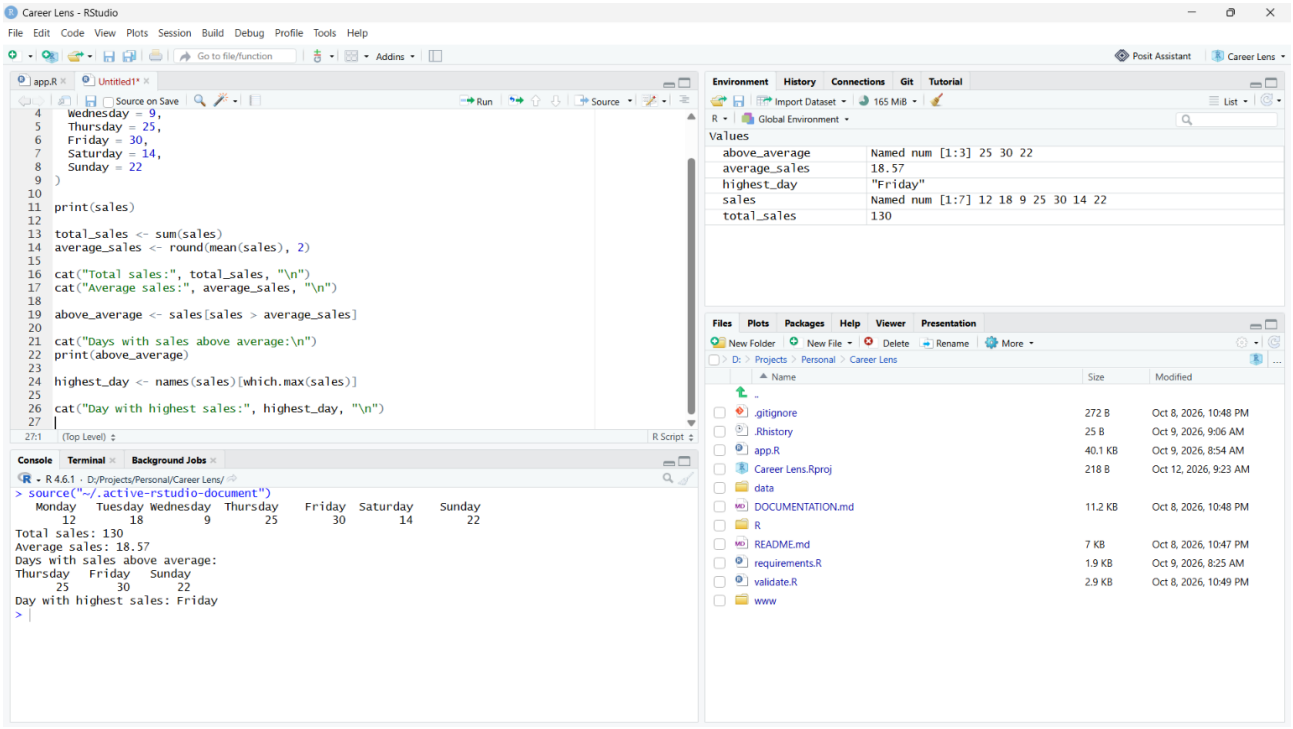
Days with sales above average:
Thursday   Friday   Sunday
      25       30       22

Day with highest sales: Friday
```

1.6 Result

The weekly sales total is 130 units and the average is 18.57 units. Thursday, Friday, and Sunday have sales above the weekly average. Friday has the highest sales with 30 units.

Screenshot Space – Question 1 RStudio Output



Question 2: Lists – Student Record

2.1 Problem Understanding

A student record is required for Asha. The record contains a character value for the name, a numeric vector containing three test marks (78, 85, 92), and a logical value indicating that the student has passed.

Input: Name = Asha, marks = c(78, 85, 92), passed = TRUE.

Output: Student list, average marks, grade, number of list elements, and element names.

Constraints: A single list must store all fields; elements must be accessed using $ or [[ ]]; the grade must be added by assignment without rebuilding the list; no data frame may be used.

2.2 Algorithm

- Create the list student with name, marks, and passed fields.
- Access the marks through student$marks and calculate their mean.
- Use a condition to assign grade A when the average is at least 85; otherwise assign B.
- Add grade directly using student$grade <- ... without rebuilding the list.
- Use length(student) and names(student) to display the number and names of elements.

2.3 Pseudocode

```
START
Create list student with name, marks, and passed
average_marks ← mean(student$marks)
IF average_marks >= 85 THEN
    student$grade ← "A"
ELSE
    student$grade ← "B"
END IF
Display student
Display length(student) and names(student)
STOP
```

2.4 R Program

```
# Question 2: Student Record

student <- list(
  name = "Asha",
  marks = c(78, 85, 92),
  passed = TRUE
)

print(student)

average_marks <- mean(student$marks)

cat("Average marks:", average_marks, "\n")

student$grade <- ifelse(average_marks >= 85, "A", "B")

cat("Grade:", student$grade, "\n")

cat("Number of elements:", length(student), "\n")
cat("Names of elements:\n")
print(names(student))
```

2.5 Expected Output

```
$name
[1] "Asha"

$marks
[1] 78 85 92

$passed
[1] TRUE

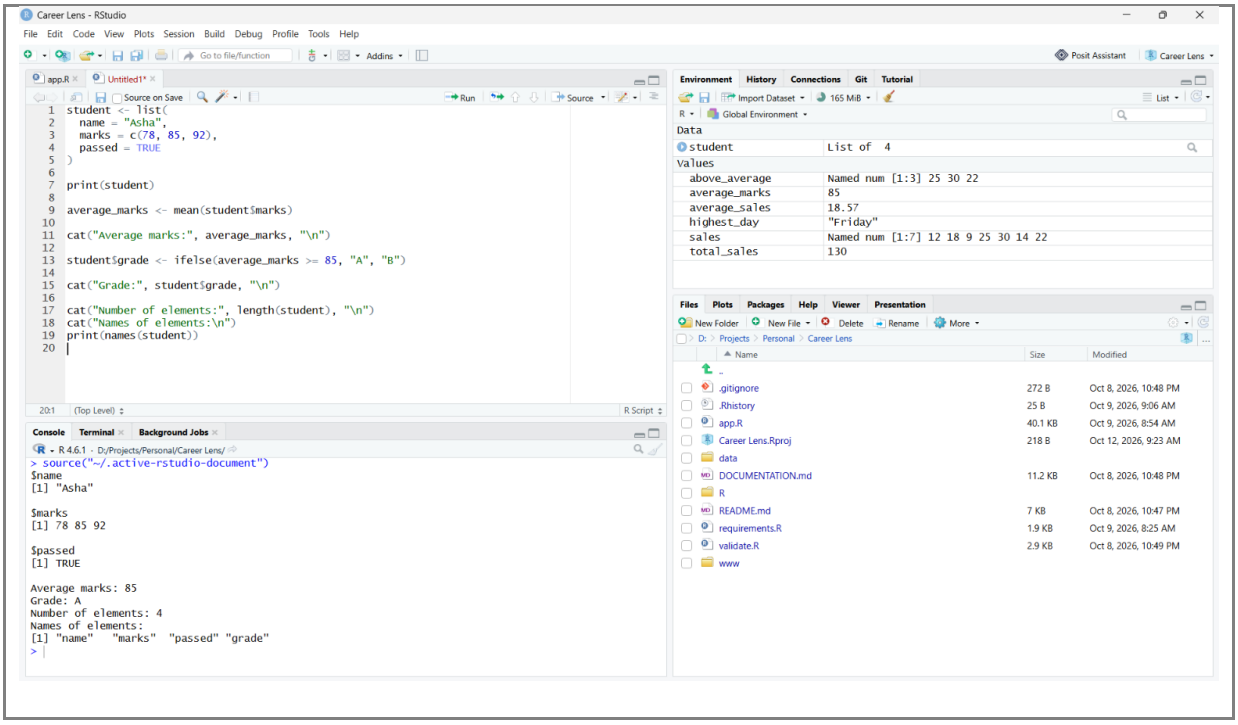
Average marks: 85
Grade: A
Number of elements: 4

Names of elements:
[1] "name" "marks" "passed" "grade"
```

2.6 Result

The average mark is 85, so Asha receives grade A. The list contains four elements after the grade is added: name, marks, passed, and grade.

Screenshot Space – Question 2 RStudio Output



Question 3: Matrices – Operations on Two Matrices

3.1 Problem Understanding

Matrix A contains the numbers 1 to 9 arranged row-wise in a 3 × 3 matrix. Matrix B contains the row-wise values 1, 0, 2, 0, 1, 0, 3, 0, 1.

Output: Matrices A and B, element-wise product, matrix product, transpose of A, and row sums of A.

Constraints: Both matrices must be created using matrix() with byrow = TRUE; loops are prohibited; * must be used for element-wise multiplication and %*% for matrix multiplication; only base R functions may be used.

3.2 Algorithm

- Create matrix A using 1:9 with three rows, three columns, and byrow = TRUE.
- Create matrix B using the given nine values with three rows, three columns, and byrow = TRUE.
- Calculate A * B for element-wise multiplication.
- Calculate A %*% B for matrix multiplication.
- Calculate t(A) for the transpose.
- Calculate rowSums(A) for row totals.

3.3 Pseudocode

```
START
Create A using values 1 to 9, 3 × 3, by row
Create B using given values, 3 × 3, by row
element_product ← A * B
matrix_product ← A %*% B
transpose_A ← transpose of A
row_sum_A ← row sums of A
Display all results
STOP
```

3.4 R Program

```
# Question 3: Operations on Two Matrices

A <- matrix(1:9, nrow = 3, ncol = 3, byrow = TRUE)

B <- matrix(
  c(1, 0, 2,
    0, 1, 0,
    3, 0, 1),
  nrow = 3,
  ncol = 3,
  byrow = TRUE
)

cat("Matrix A:\n")
print(A)

cat("Matrix B:\n")
print(B)

element_product <- A * B

cat("Element-wise product of A and B:\n")
print(element_product)

matrix_product <- A %*% B

cat("Matrix product of A and B:\n")
print(matrix_product)

transpose_A <- t(A)

cat("Transpose of A:\n")
print(transpose_A)

row_sum_A <- rowSums(A)

cat("Row sums of A:\n")
print(row_sum_A)
```

3.5 Expected Output

```
Matrix A:
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9

Matrix B:
     [,1] [,2] [,3]
[1,]    1    0    2
[2,]    0    1    0
[3,]    3    0    1

Element-wise product:
     [,1] [,2] [,3]
[1,]    1    0    6
[2,]    0    5    0
[3,]   21    0    9

Matrix product:
     [,1] [,2] [,3]
[1,]   10    2    5
[2,]   22    5   14
[3,]   34    8   23

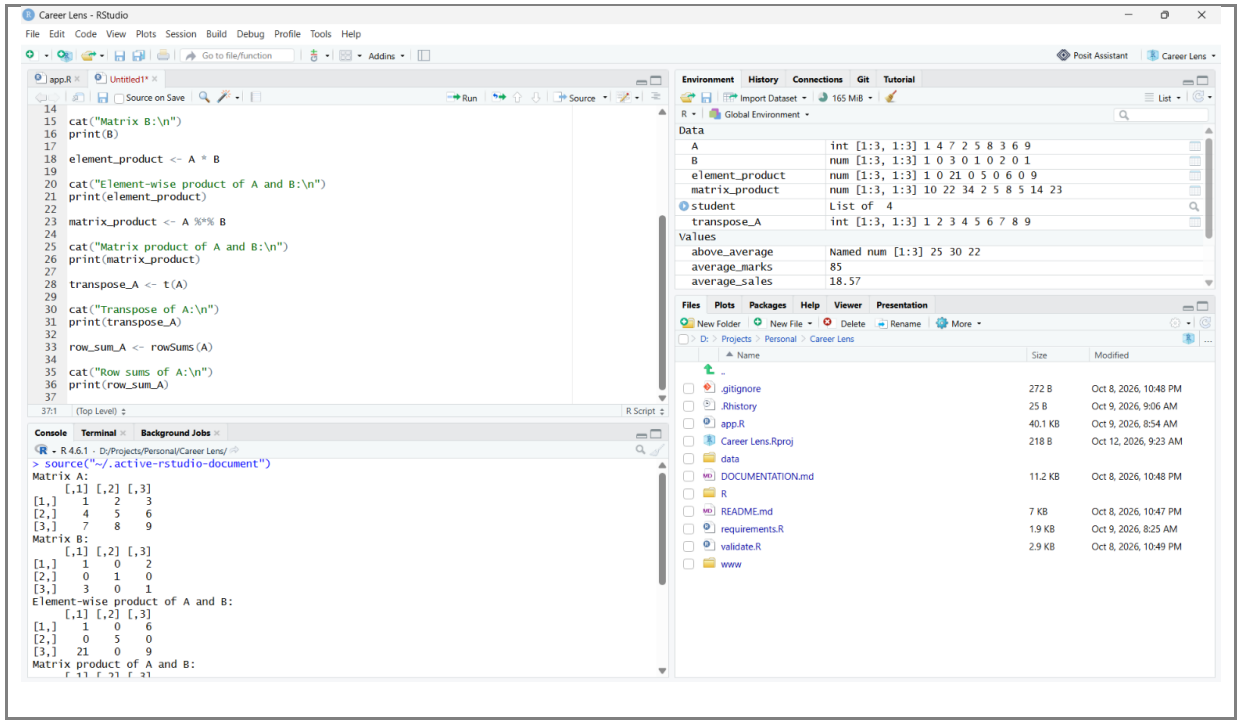
Transpose of A:
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9

Row sums of A:
[1]  6 15 24
```

3.6 Result

The two 3 × 3 matrices were created successfully. Element-wise multiplication and matrix multiplication were performed using the required operators. The transpose and row sums of A were also obtained successfully.

Screenshot Space – Question 3 RStudio Output



Question 4: Arrays – Three-Dimensional Data

4.1 Problem Understanding

A warehouse stores 12 item counts in a three-dimensional array with 2 rows, 3 columns, and 2 layers. The values stored are 1 to 12.

Input: Values 1 through 12 and dimensions c(2, 3, 2).

Output: The complete second layer, the element at row 2 column 3 of layer 1, and the sum of each layer.

Constraints: array() must be used with the dim argument; no loops may be used; extraction must use comma-separated indexing; apply() must be used to calculate layer totals.

4.2 Algorithm

- Create arr using array(1:12, dim = c(2, 3, 2)).
- Extract the second layer using arr[, , 2].
- Extract row 2, column 3, layer 1 using arr[2, 3, 1].
- Use apply(arr, 3, sum) to calculate the total of each layer.
- Display the complete array and all requested results.

4.3 Pseudocode

```
START
Create arr containing values 1 to 12 with dimensions 2 × 3 × 2
second_layer ← arr[, , 2]
element ← arr[2, 3, 1]
layer_totals ← apply(arr, 3, sum)
Display second_layer, element, and layer_totals
STOP
```

4.4 R Program

```
# Question 4: Three-Dimensional Array

arr <- array(
  1:12,
  dim = c(2, 3, 2)
)

cat("3-dimensional array:\n")
print(arr)

cat("Second layer:\n")
print(arr[, , 2])

element <- arr[2, 3, 1]

cat("Element at row 2, column 3, layer 1:", element, "\n")

layer_totals <- apply(arr, 3, sum)

cat("Sum of each layer:\n")
print(layer_totals)
```

4.5 Expected Output

```
, , 1

     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

, , 2

     [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12

Second layer:
     [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12

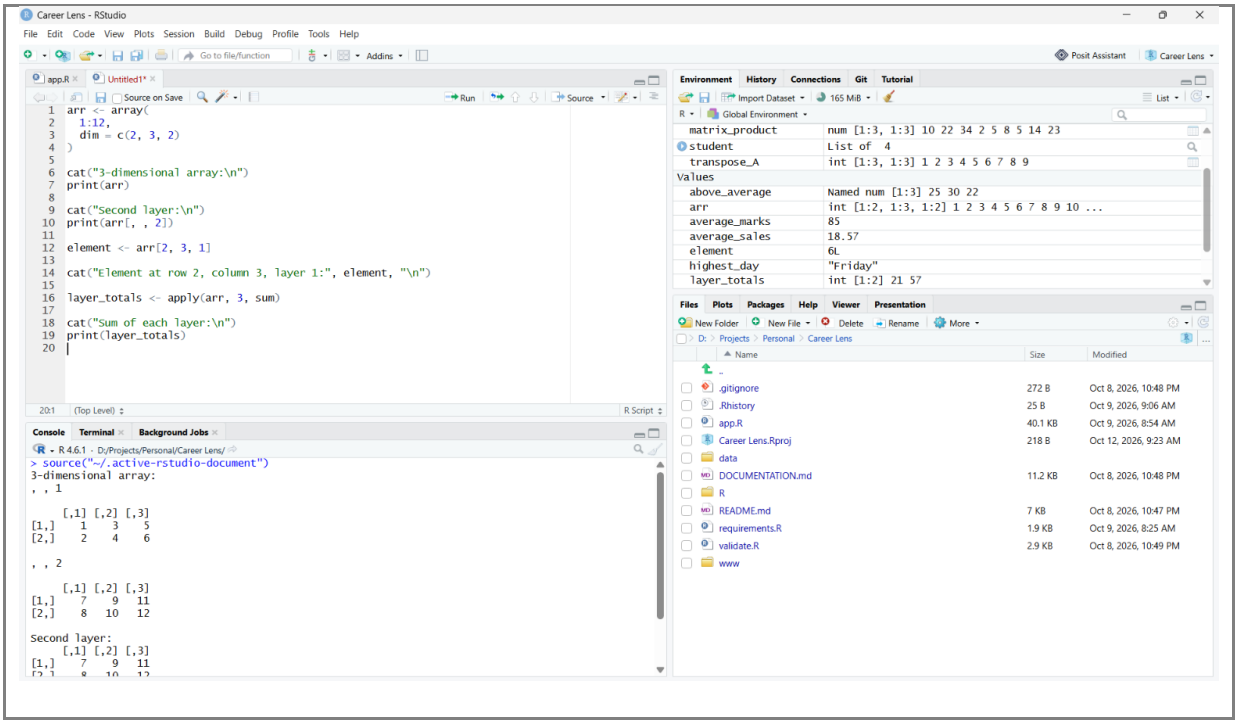
Element at row 2, column 3, layer 1: 6

Sum of each layer:
[1] 21 57
```

4.6 Result

The three-dimensional array was successfully created. The second layer contains values 7 to 12 in the specified arrangement. The requested element is 6, and the two layer totals are 21 and 57.

Screenshot Space – Question 4 RStudio Output



Question 5: Class Types and Coercion – Price List

5.1 Problem Understanding

Prices read from a text file are initially stored as character values: 120, 85.5, and 200. The shop needs the total bill and the total after a 10% discount.

Input: Character vector c("120", "85.5", "200").

Output: Class before conversion, numeric vector and class after conversion, total bill, discounted bill, and the class and explanation of c(1, "a", TRUE).

Constraints: The existing character vector must be converted rather than retyped as numbers; as.numeric() must be used; class() must be checked before and after conversion; loops are prohibited.

5.2 Algorithm

- Create the existing prices as a character vector.
- Use class(prices) to verify that the initial class is character.
- Convert the vector using as.numeric(prices).
- Use class(prices_numeric) to verify that the converted class is numeric.
- Calculate the total using sum(prices_numeric).
- Calculate the discounted total as total × 0.90.
- Create c(1, "a", TRUE), check its class, and explain R's automatic coercion to character.

5.3 Pseudocode

```
START
Create character vector prices
Display class(prices)
prices_numeric ← as.numeric(prices)
Display class(prices_numeric)
total ← sum(prices_numeric)
discounted_total ← total × 0.90
Create mixed_vector ← c(1, "a", TRUE)
Display class(mixed_vector)
Explain character coercion
STOP
```

5.4 R Program

```
# Question 5: Class Types and Coercion

prices <- c("120", "85.5", "200")

cat("Prices:\n")
print(prices)

cat("Class before conversion:", class(prices), "\n")

prices_numeric <- as.numeric(prices)

cat("Prices after conversion:\n")
print(prices_numeric)

cat("Class after conversion:", class(prices_numeric), "\n")

total <- sum(prices_numeric)
discounted_total <- total * 0.90

cat("Total bill:", total, "\n")
cat("Bill after 10% discount:", discounted_total, "\n")

mixed_vector <- c(1, "a", TRUE)

cat("Mixed vector:\n")
print(mixed_vector)

cat("Class of mixed vector:", class(mixed_vector), "\n")
```

5.5 Expected Output

```
Prices:
[1] "120"  "85.5" "200"

Class before conversion: character

Prices after conversion:
[1] 120.0  85.5 200.0

Class after conversion: numeric

Total bill: 405.5
Bill after 10% discount: 364.95

Mixed vector:
```

```
[1] "1"    "a"    "TRUE"

Class of mixed vector: character
```

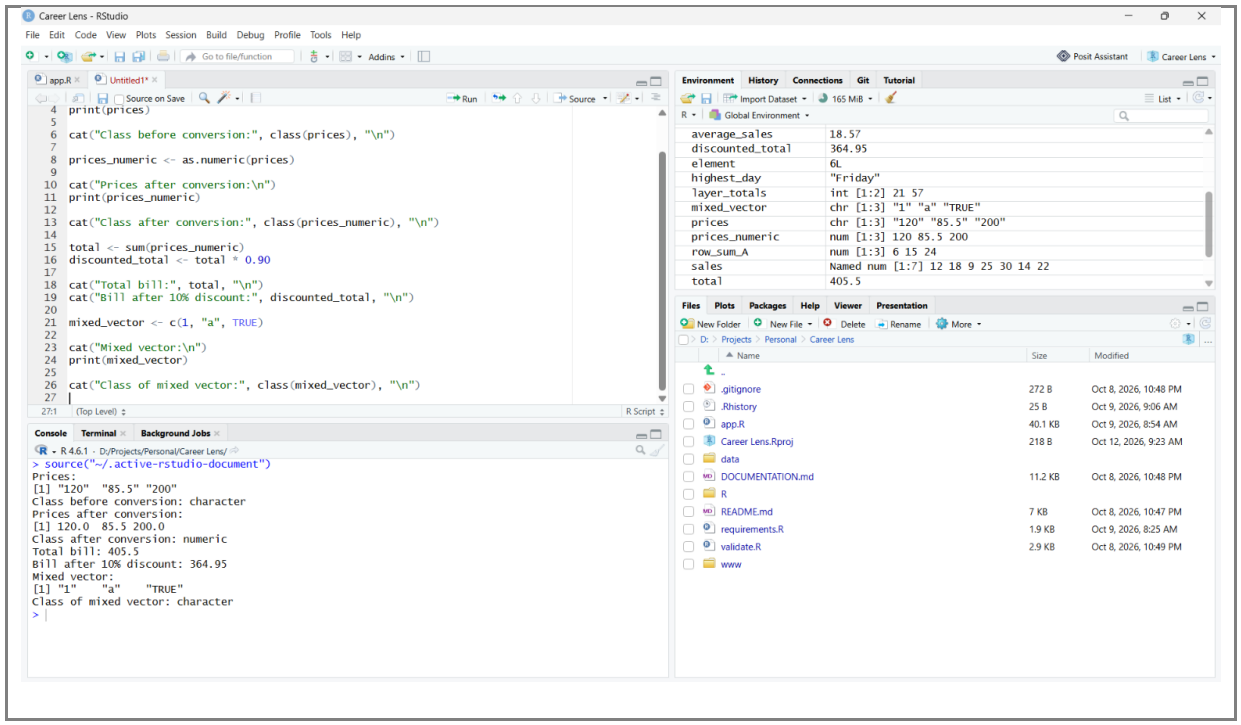
5.6 Explanation of Coercion

R vectors are homogeneous, meaning all elements in a vector are generally stored using a common data type. In c(1, "a", TRUE), the numeric value 1 and logical value TRUE are converted to character values because character is the common compatible type required to store all three elements in one vector. Therefore, the resulting vector is "1", "a", "TRUE" and its class is character.

5.7 Result

The character price vector was converted successfully using as.numeric(). The total bill is 405.50 and the bill after a 10% discount is 364.95. The mixed vector demonstrates automatic coercion to the character class.

Screenshot Space – Question 5 RStudio Output



Final Result

All five constraint-based R programming problems have been solved using appropriate base R constructs. The submission demonstrates understanding of vectors, lists, matrices, arrays, class types, coercion, vectorized operations, indexing, conditional logic, and apply(). The solutions are complete and aligned with the stated assessment constraints and exemplary-level evaluation criteria.